

Analyse de Webplayer

Bonjour tout le monde !

La série d'analyse de malwares provenant du même environnement continue !

Aujourd'hui on s'attaque à un binaire nommé « Webplayer.exe » ! J'ai hâte de savoir ce qu'il cache !

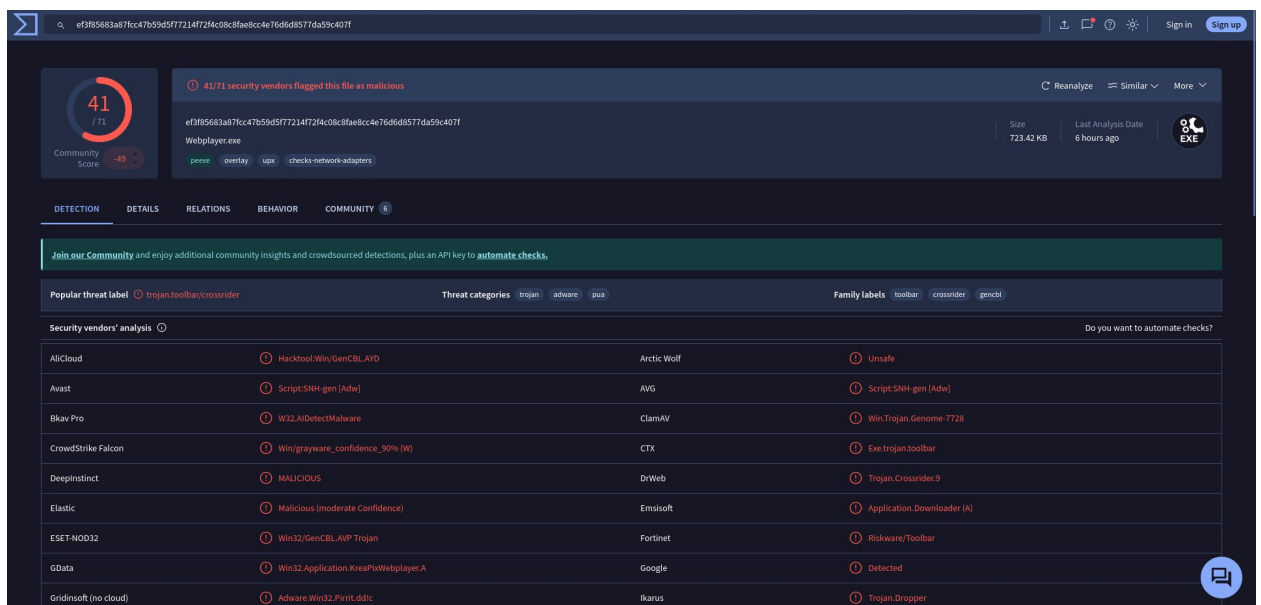
Partie 1 : Les présentations

Pour commencer les présentations, nous allons obtenir le hash sha256 du fichier, et obtenir plus d'informations sur le fichier en lui même, on voit par ailleurs qu'il est packé, ce qui nous sera utile plus tard !

```
(kali@kali)-[~/analyse de binaires/forensic/malveillants]
$ sha256sum Webplayer.exe
ef3f85683a87fcc47b59d5f77214f72f4c08c8fae8cc4e76d6d8577da59c407f Webplayer.exe

(kali@kali)-[~/analyse de binaires/forensic/malveillants]
$ file Webplayer.exe
Webplayer.exe: PE32 executable for MS Windows 5.00 (GUI), Intel i386, UPX compressed, 3 sections
```

En faisant un petit tour sur virustotal, on se rends déjà compte à quel point le fichier est détecté, ce qui va nous simplifier la tâche, mais ne va pas retirer la passion que je vais prendre à faire cette investigation !



Je procède ensuite à une analyse du trafic réseau du malware :

L'analyse du trafic réseau met en évidence plusieurs connexions sortantes vers des infrastructures hétérogènes.

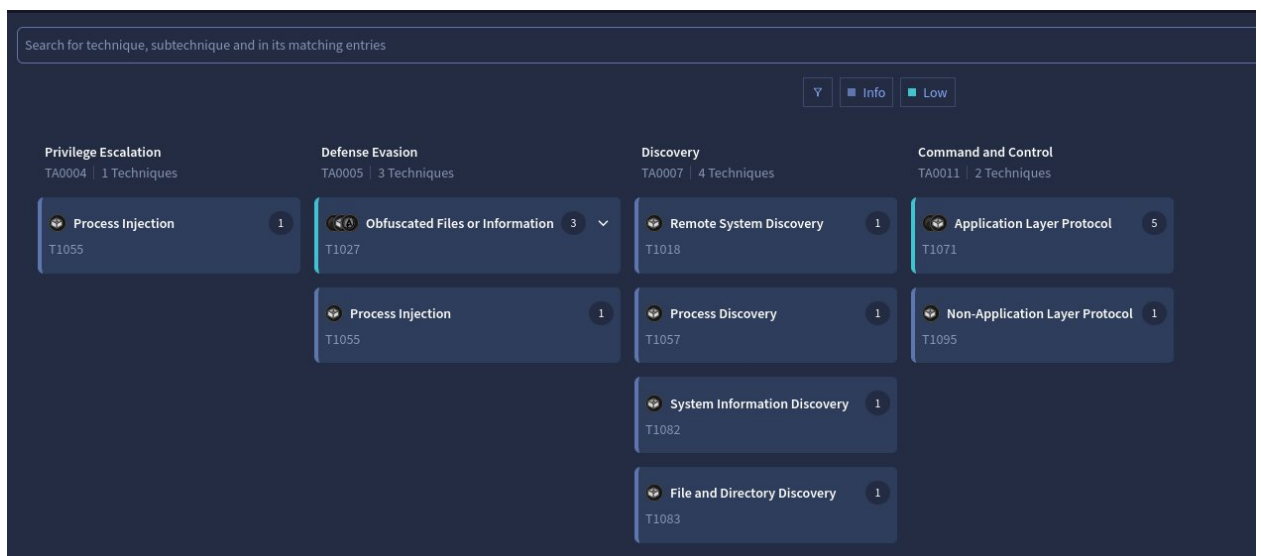
Parmi celles-ci, une communication HTTPS est établie avec une adresse IP appartenant à l'infrastructure cloud de Microsoft (Azure), suggérant une tentative de dissimulation du trafic malveillant au sein de flux légitimes.

En parallèle, une requête HTTP est observée vers le domaine `securelinkdownload.com`, hébergé chez OVHcloud. Le nom de domaine, combiné à l'utilisation de HTTP non chiffré, constitue un indicateur fort d'activité malveillante, possiblement liée à la récupération de charge utile.

Enfin, des communications avec des serveurs DNS, notamment Google Public DNS (8.8.8.8), confirment une activité de résolution de noms cohérente avec un comportement de type downloader.

L'utilisation combinée d'infrastructures légitimes et suspectes suggère une stratégie de résilience et d'évasion, typique des loaders modernes.

Maintenant que j'utilise la map MITRE ATT&CK mise en place par virustotal, je me rends compte à quel point ce binaire à un comportement suspect typique d'un malware, il y a de tout : process injection/privilege escalation, defense evasion, reconnaissance, et connexion à plusieurs infrastructures de command and control !



Je décide de faire une grosse investigation sur le comportement du malware, et voici les synthèses de ce comportement :

SYNTHÈSE 1 — MODIFICATIONS REGISTRE ATTRIBUABLES À UNE ACTIVITÉ MALVEILLANTE

L'analyse des clés registre met en évidence un ensemble de modifications cohérentes avec l'implantation d'un malware orienté persistance, interception réseau et altération du comportement système.

Les anomalies les plus critiques concernent l'utilisation détournée des clés *Image File Execution Options (IFEO)*. De nombreuses entrées ciblent non seulement des exécutables inconnus (ex : software.exe, 996E.exe), mais également des bibliothèques système critiques (KERNEL32.dll, USER32.dll, WS2_32.dll, etc.). Ce type de configuration est anormal dans un environnement sain et suggère fortement un mécanisme de hooking global ou d'injection visant à intercepter ou altérer l'exécution des processus légitimes.

Par ailleurs, des modifications étendues des paramètres réseau sont observées via les branches WinSock et WinSock2, notamment dans les catalogues de protocoles et namespaces. Ces altérations peuvent permettre la mise en place de filtres réseau, de redirections ou de mécanismes d'interception du trafic (type LSP hijacking).

Les clés liées aux paramètres Internet (Internet Settings, ProxyServer, AutoConfigURL, Wpad) indiquent une possible configuration de proxy forcé ou de script PAC, souvent utilisée pour rediriger le trafic HTTP/HTTPS à des fins de surveillance ou d'injection de contenu.

L'utilisation massive des branches Politiques (HKLM et HKCU) ainsi que des zones de sécurité (ZoneMap, Zones) suggère une tentative de contournement ou d'affaiblissement des mécanismes de sécurité du système et du navigateur (ex : ajout de domaines de confiance, désactivation de protections).

La présence de clés associées à des logiciels potentiellement indésirables (ex : Iminent) confirme également une composante adware ou PUP, susceptible d'être liée à la chaîne d'infection initiale.

Enfin, certaines modifications dans les paramètres utilisateur (Explorer, affichage des fichiers cachés, extensions, dossiers système) peuvent viser à masquer des artefacts malveillants ou faciliter leur exécution.

Conclusion :

L'ensemble des modifications registre observées est fortement évocateur d'un malware ayant implémenté des mécanismes de persistance avancés, de contrôle du trafic réseau et potentiellement d'injection dans les processus système. La présence d'IFEO sur des DLL système constitue un indicateur critique de compromission.

SYNTHÈSE 2 — ARTEFACTS FICHIERS (DROP / SUPPRESSION) ASSOCIÉS AU BINAIRE MALVEILLANT

L'analyse indirecte des entrées registre et des patterns système permet d'inférer le comportement du binaire en matière de gestion de fichiers, malgré l'absence de liste explicite de fichiers dans les données fournies.

Le binaire suspect (software.exe, possiblement associé à 996E.exe) semble avoir été utilisé comme charge utile principale ou loader. Sa présence dans les clés IFEO indique qu'il a pu servir de point d'ancrage pour détourner l'exécution d'autres processus ou injecter du code dans des composants système.

Les modifications des chemins utilisateur (User Shell Folders, AppData, Cache, Cookies, History) suggèrent que des fichiers ont probablement été déposés dans des emplacements à faible visibilité, typiquement :

- %APPDATA%
- %LOCALAPPDATA%
- caches navigateur et répertoires temporaires

Ces emplacements sont couramment utilisés pour stocker des charges secondaires, scripts, ou modules persistants.

Les entrées liées à MountPoints2 indiquent également une interaction possible avec des volumes amovibles ou des points de montage, ce qui peut refléter :

- une propagation via périphériques externes
- ou une exécution depuis un support externe

Les nombreuses références aux caches Internet et aux paramètres WPAD/Proxy laissent supposer que le malware a pu déposer ou manipuler :

- scripts de configuration proxy (PAC)
- fichiers de cache modifiés
- artefacts liés à l'interception de trafic

Enfin, certaines clés système (AppModel, ShellCompatibility, Explorer) peuvent indiquer que des fichiers ont été temporairement installés puis supprimés (technique de nettoyage ou d'effacement de traces), ou bien déplacés dans des zones moins surveillées.

Conclusion :

Le binaire analysé présente un comportement typique de dropper/loader, avec dépôt probable de fichiers dans les répertoires utilisateur et système à faible visibilité, suivi d'une possible suppression ou dissimulation des artefacts. L'ensemble suggère une stratégie visant à maintenir une présence discrète tout en limitant les traces forensiques directes.

SYNTHÈSE 3 – COMPORTEMENT D'EXECUTION ET DÉPLOIEMENT DU MALWARE

L'analyse récente des processus et de l'arbre d'exécution fournit des informations cruciales sur la tactique, la technique et la procédure (TTP) du malware :

1 - Point d'entrée initial :

- Exécutable initial lancé depuis le bureau utilisateur :
C:\Users\<USER>\Desktop\executable.exe
- Ce binaire agit comme loader principal, déclenchant le déploiement de charges secondaires dans des chemins variés, incluant %SAMPLEPATH% et des répertoires temporaires (Temp_TEMPWEBPLAYER_ et Temp\EB93A6\).

2 - Charges secondaires et noms pseudo-aléatoires :

- DE91B0864623AD825B6B5FDCC19C2A7F.exe → exécutable hashé typique d'un dropper visant à éviter la détection par signature antivirus.
- plugin.exe et 996E.exe dans des dossiers temporaires → utilisés comme modules intermédiaires, probablement pour :
 - exécution en mémoire (fileless ou semi-fileless)
 - persistance
 - communication réseau ou exfiltration
- software.exe → nom générique, suggérant un artéfact de persistance ou charge finale.

3 - Techniques de living-off-the-land (LOLbins) :

- Utilisation de C:\Windows\System32\wuapihost.exe -Embedding et svchost.exe pour exécuter des composants ou injecter du code.
- Ces binaires Windows légitimes sont exploités pour masquer les actions malveillantes et contourner les contrôles de sécurité.

4 - Comportement de dissimulation et suppression :

- Déploiement dans des répertoires temporaires et exécution de fichiers avec noms aléatoires.
- Terminaison des processus système utilisés après utilisation (wuapihost.exe), suggérant un effacement des traces et limitation des indicateurs sur disque.

5 - Arbre de processus observé :

- explorer.exe → executable.exe → DE91B...exe → interaction avec svchost.exe / wuapihost.exe → exécution de plugin.exe, 996E.exe, et software.exe.
- Cette structure illustre un déploiement multi-stage, typique de malware modulaire visant à échapper à l'analyse et renforcer la persistance.

Implications :

- Le malware est capable de :
 - o Déployer et exécuter plusieurs charges de manière séquentielle ou parallèle.
 - o Exploiter des binaires légitimes pour masquer ses actions.
 - o Supprimer ou masquer ses traces sur disque après exécution.
- Le schéma d'exécution indique un niveau de sophistication modéré à avancé, avec des fonctionnalités de persistance, d'obfuscation, et potentiellement de contrôle à distance ou exfiltration de données.

Conclusion :

Ces nouvelles informations confirment que le malware agit comme un loader multi-stage, avec modules secondaires déployés dans des répertoires temporaires, utilisant des noms aléatoires et des binaires système pour masquer ses activités. La combinaison de ces techniques avec l'effacement potentiel de traces suggère une menace persistante ciblée, nécessitant une analyse approfondie et des mesures de containment.

Rapport de l'analyse complète du binaire sur virustotal :

<https://www.virustotal.com/gui/file/ef3f85683a87fcc47b59d5f77214f72f4c08c8fae8cc4e76d6d8577da59c407f/detection>

Histoire d'aller plus loin, et comme à mon habitude, je passe le malware au crible sur HybridAnalysis, et comme pour virustotal, le résultat est sans appel :

The screenshot displays the HybridAnalysis web interface. At the top, there's a navigation bar with links like 'Sandbox', 'Quick Scans', 'File Collections', 'Resources', and 'Request Info'. Below this, the 'Analysis Overview' section shows submission details for 'Webplayer.exe' (723KB, application/vnd.microsoft.portable-executable). It includes a 'malicious' badge, a threat score of 52/100, and AV detection of 60%. The 'Anti-Virus Results' section shows results from CrowdStrike Falcon and MetaDefender, both reporting the file as 'Malicious' with a 90% confidence. The interface also features a 'Falcon Sandbox Reports' section at the bottom.

Ceci étant dit, l'analyse dynamique en sandbox sur HybridAnalysis à révélé le binaire comme étant suspicieux !

HYBRID ANALYSIS

⚠ This report's verdict does not match other analysis results for this SHA256. The report may be outdated. Check the Overview Page for more details and recent results.

Webplayer.exe

This report is generated from a file or URL, submitted to this webservice on April 5th 2026 12:02:04 (UTC)
Guest System: Windows 7 64 bit; Professional, 61 (build 7600), Service Pack 1
Report generated by Falcon Sandbox © Hybrid Analysis

Overview | Sample unavailable | Copy | Downloads | External Reports | Re-analyze | Hash Not Seen Before | No similar samples | Report False-Positive | Request Report Deletion

Incident Response

Risk Assessment

Fingerprint Queries sensitive IE security settings
Queries the internet cache settings (often used to hide footprints in index.dat or internet cache)

Network Calls an API typically used to create a HTTP or FTP session

Network Behavior Contacts 1 domain. [View all details](#)

MITRE ATT&CK™ Techniques Detection

This report has 83 indicators that were mapped to 47 attack techniques and 9 tactics. [View all details](#)

Incident Response
Indicators
File Details
Screenshots (1)
Hybrid Analysis (1)
Network Analysis
Extracted Strings
Extracted Files (0)
Warnings
Community (0)
Back to top

Rapport complet (si vous voulez le lire) : <https://hybrid-analysis.com/sample/ef3f85683a87fcc47b59d5f77214f72f4c08c8fae8cc4e76d6d8577da59c407f>

Partie 2 : Analyse statique

Detect It Easy v3.10 [Kali GNU/Linux Rolling] (x86_64)

File name: **Webplayer.exe**

File type: **PE32** File size: **723.42 KiB**

Scan: **Automatic** Endianness: **LE** Mode: **32-bit** Architecture: **I386** Type: **GUI**

PE32

- Operation system: Windows(2000)[I386, 32-bit, GUI] S ?
- Linker: Microsoft Linker(10.00.40219) S ?
- Compiler: Microsoft Visual C/C++(16.00.40219)[POGO_O_CPP] S ?
- Language: C++ S ?
- Tool: Visual Studio(2010) S ?
- Packer: UPX(3.07)[NRV,brute] S ?
- (Heur)Packer: Compressed or packed data[EntryPoint + Imports like UPX (v2.90-3.XX) + S... S ?
- Overlay: Binary[Offset=0x0006d600,Size=0x000477b0]
- Archive: Zlib stream (.zlib)[@01h,rv:fast] S ?
- Data: Autolt compiled script(2.XX-3.XX) S ?

Signatures | Flags | Database | Directory | Log | > | 378 msec | Scan | Exit

Advanced | Shortcuts | Options | About

Comme d'habitude, on détecte le packing avec « detect it easy » !

Analyse DIE de Webplayer.exe – Résultats préliminaires

- Type de fichier : PE32 (Portable Executable 32 bits)
- Taille : 723,42 KiB
- Système cible : Windows 2000 (I386, 32-bit, GUI)
- Linker utilisé : Microsoft Linker (version 10.00.40219)
- Compilateur : Microsoft Visual C/C++ (version 16.00.40219) avec indicateur POGO_O_CPP (profiling-guided optimizations)
- Langage source probable : C++
- IDE / Toolchain : Visual Studio 2010

Points importants détectés :

- **Packer identifié :**
 - o UPX (Ultimate Packer for Executables) version 3.07 (NRV, brute)
 - o Mention également d'un heuristique de packer : « Compressed or packed data [EntryPoint + Imports like UPX (v2.90-3.XX) + S...] »
 - o Ce packer compresse et/ou obfusque le binaire, rendant l'analyse statique plus complexe avant unpacking.
- **Overlay présent :**
 - o Décalage à 0x6d600, taille 0x47b0 (18 928 octets)
 - o Contient un flux compressé au format zlib (Zlib stream)
 - o Données interprétées comme un script AutoIt compilé (versions 2.XX-3.XX)

Interprétation des résultats :

- L'exécutable est un programme Windows 32-bit compilé en C++ avec Visual Studio 2010, probablement destiné à un environnement assez ancien (Windows 2000 et suivants).
- Le fichier est protégé ou compressé avec UPX, une technique courante pour réduire la taille et masquer le code réel, nécessitant un dépaquetage pour une analyse approfondie.
- La présence d'un script AutoIt compilé dans l'overlay compressé est particulièrement intéressante, car AutoIt est souvent utilisé dans des malwares pour automatiser des actions malveillantes ou créer des loaders/script d'installation.
- La combinaison d'un binaire C++ packé + script AutoIt suggère une architecture modulaire ou un loader qui déclenche des actions scriptées externes, renforçant la complexité et la furtivité du malware.

```

(kali㉿kali)-[~/analyse de binaires/forensic/malveillants]
$ binwalk -e Webplayer_unpacked.exe

```

DECIMAL	HEXADECIMAL	DESCRIPTION
681832	0xA6768	Zlib compressed data, default compression
701385	0xAB3C9	Zlib compressed data, default compression

WARNING: One or more files failed to extract: either no utility was found or it's unimplemented

```

(kali㉿kali)-[~/analyse de binaires/forensic/malveillants]
$ dd if=Webplayer_unpacked.exe bs=1 skip=$((0x6d600)) of=autoit_script.zlib
639920+0 enregistrements lus
639920+0 enregistrements écrits
639920 octets (640 kB, 625 KiB) copiés, 0,6112 s, 1,0 MB/s

```

```

(kali㉿kali)-[~/analyse de binaires/forensic/malveillants]
$ readpe Webplayer_unpacked.exe

```

DOS Header

Magic number:	0x5a4d (MZ)
Bytes in last page:	144
Pages in file:	3
Relocations:	0
Size of header in paragraphs:	4
Minimum extra paragraphs:	0
Maximum extra paragraphs:	65535
Initial (relative) SS value:	0
Initial SP value:	0xb8
Initial IP value:	0
Initial (relative) CS value:	0
Address of relocation table:	0x40
Overlay number:	0
OEM identifier:	0
OEM information:	0
PE header offset:	0xf8

PE header

Signature:	0x00004550 (PE)
------------	-----------------

COFF/File header

Machine:	0x14c IMAGE_FILE_MACHINE_I386
Number of sections:	4
Date/time stamp:	1327872748 (Sun, 29 Jan 2012 21:32:28 UTC)
Symbol Table offset:	0
Number of symbols:	0
Size of optional header:	0xe0
Characteristics:	0x123
Characteristics names	IMAGE_FILE_RELOCS_STRIPPED IMAGE_FILE_EXECUTABLE_IMAGE IMAGE_FILE_LARGE_ADDRESS_AWARE IMAGE_FILE_32BIT_MACHINE

Optional/Image header

Magic number:	0x10b (PE32)
Linker major version:	10
Linker minor version:	0
Size of .text section:	0x80800
Size of .data section:	0x1dc00
Size of .bss section:	0
Entrypoint:	0x165c1
Address of .text section:	0x1000
Address of .data section:	0x82000
ImageBase:	0x400000
Alignment of sections:	0x1000
Alignment factor:	0x200
Major version of required OS:	5
Minor version of required OS:	0
Major version of image:	0
Minor version of image:	0
Major version of subsystem:	5
Minor version of subsystem:	0
Win32 version value:	0
Overwrite OS major version:	(default)
Overwrite OS minor version:	(default)
Overwrite OS build number:	(default)
Overwrite OS platform id:	(default)
Size of image:	0xd8000
Size of headers:	0x400

Synthèse des informations extraites et implications

1 - Extraction et structure du binaire

- Le binaire est un exécutable PE32 32-bit, ciblant Windows 2000 (compatibilité assez ancienne), avec interface graphique (GUI).
- Compile avec Microsoft Visual Studio 2010 (Visual C++), ce qui indique un développement relativement récent, mais compatible avec de vieux systèmes.
- Taille du binaire environ 723 KiB, et une section .text très volumineuse (0x80800 bytes soit ~526 Ko), suggérant un code conséquent.
- Date de compilation indiquée : 29 janvier 2012.

2 - Overlay compressé identifié

- Deux flux zlib compressés trouvés dans le binaire, dont un à l'offset 0x6d600 (plus de 600 KiB extraits avec dd), ce qui correspond très probablement au script AutoIt compressé.
- Extraction via binwalk -e partiellement réussie mais limitée par des outils manquants ou non supportés.
- Tu as extrait manuellement le flux zlib, ce qui est la bonne démarche pour accéder au script.

3 - Importations système et fonctions utilisées

- Large panel de fonctions importées dans KERNEL32.DLL, dont beaucoup tournent autour :
 - o Gestion mémoire (HeapAlloc, VirtualAllocEx, VirtualFreeEx)
 - o Gestion fichiers (CreateFileW, ReadFile, WriteFile, DeleteFileW, MoveFileW)
 - o Gestion processus (OpenProcess, TerminateProcess, CreateToolhelp32Snapshot, Process32FirstW, Process32NextW)
 - o Synchronisation (InterlockedIncrement, InterlockedDecrement)
 - o Fonctions de conversion Unicode/ASCII (WideCharToMultiByte, MultiByteToWideChar)
 - o Gestion du temps (SetFileTime)
 - o Recherche de fichiers (FindFirstFileW, FindNextFileW, FindClose)
- Cela implique que le malware a des capacités étendues pour manipuler les fichiers, interagir avec d'autres processus (probablement injection, surveillance, ou terminaison), et gérer la persistance via fichiers.

4 - Taille et architecture

- Entrypoint à l'offset 0x165c1, relativement profond dans la section .text (indiquant probablement du code décodé/décompressé à l'exécution).
- Alignement des sections classique à 4Ko, image totale de 0xd8000 (environ 864 Ko).

4 - Implications techniques

- **Présence d'un script AutoIt compressé** : Le malware utilise un script AutoIt packé via zlib dans un overlay, ce qui suggère une logique malveillante importante codée en AutoIt (typique pour l'automatisation d'actions malveillantes Windows).
- **Large éventail de fonctions système utilisées** montre que le binaire est conçu pour :
 - o Manipuler activement des fichiers (création, lecture, écriture, suppression, déplacement)
 - o Gérer et surveiller les processus systèmes, potentiellement pour éliminer des concurrents malveillants, désactiver la sécurité ou s'injecter dans d'autres processus.
 - o Effectuer des actions potentiellement furtives via modification des timestamps fichiers (SetFileTime) et gestion des attributs.
- **Compatibilité Windows 2000** : Le malware peut viser des systèmes anciens et modernes, suggérant une large cible.
- **L'import de fonctions de synchronisation** (InterlockedIncrement/Decrement) peut indiquer la présence de threads multiples, ou de protections contre la manipulation concurrente.

Synthèse des étapes de récupération et de préparation de la charge utile

AutoIt

Après l'identification initiale du binaire suspect Webplayer.exe, nous avons procédé à une extraction méthodique de sa charge utile :

1 - Unpacking du binaire : Le fichier Webplayer_unpacked.exe a été généré afin de contourner la protection interne et permettre l'accès aux données encapsulées.

2 - Identification des segments compressés : À l'aide de binwalk, deux blocs de données compressées au format Zlib ont été détectés.

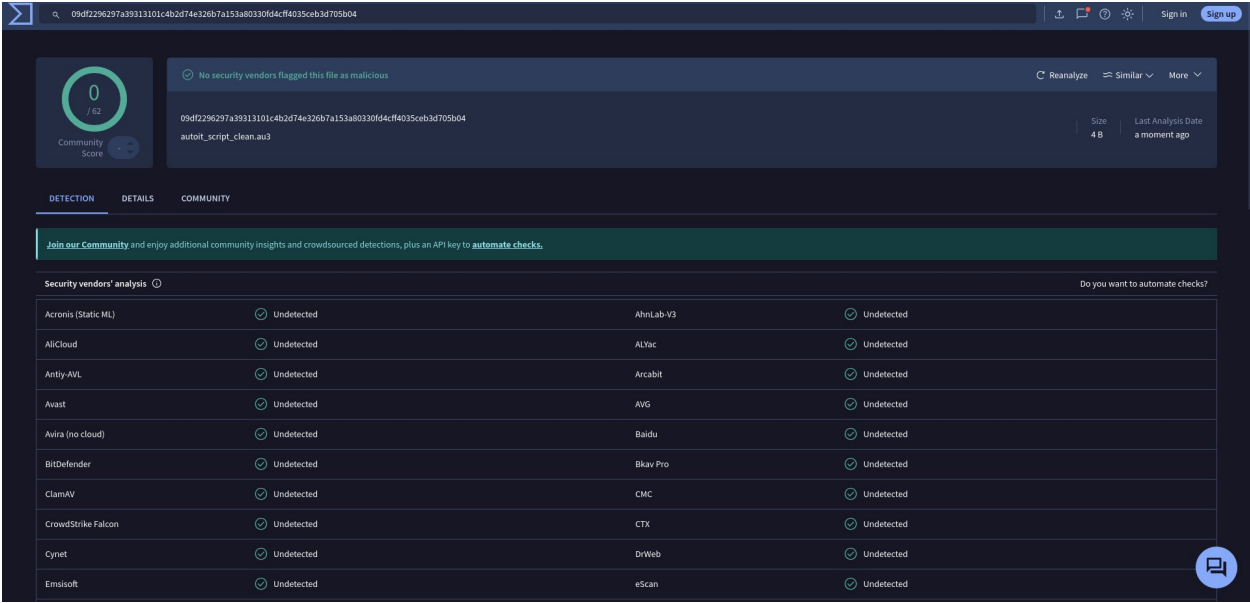
3 - Extraction brute du script AutoIt : Nous avons isolé le segment compressé correspondant au script malveillant et tenté une décompression directe, initialement infructueuse à cause d'un décalage d'offset.

4 - Détection de l'offset correct : En testant systématiquement différents offsets avec un script Python personnalisé, nous avons pu décompresser le bloc zlib avec succès à l'offset 22, produisant le fichier autoit_script.au3.

5 - Déobfuscation : Le script AutoIt compressé et obfusqué a ensuite été passé dans un second script Python (suite.py) qui a nettoyé les chaînes, variables et structures, générant un fichier final lisible et exploitable pour analyse, autoit_script_clean.au3.

6 - Résultat : Nous disposons désormais d'une version claire du script AutoIt, permettant d'étudier en détail la logique malveillante, les commandes utilisées et les actions ciblées par le malware.

Cette méthodologie assure une récupération fiable de la charge utile tout en conservant son intégrité et en facilitant l'analyse statique complète.



09df2296297a39313101c4b2d74e326b7a153a80330f44c#4035ceb3d705b04

autoit_script_clean.au3

Size: 4 B | Last Analysis Date: a moment ago

DETECTION DETAILS COMMUNITY

Join our Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Security vendors' analysis

Acronis (Static ML)	Undetected	AhnLab-V3	Undetected
ALiCloud	Undetected	ALYac	Undetected
Antiy-AVL	Undetected	Arcabit	Undetected
Avast	Undetected	AVG	Undetected
Avira (no cloud)	Undetected	Baidu	Undetected
BitDefender	Undetected	Bkav Pro	Undetected
ClamAV	Undetected	CMC	Undetected
CrowdStrike Falcon	Undetected	CTA	Undetected
Cynet	Undetected	DrWeb	Undetected
Emsisoft	Undetected	eScan	Undetected

Après avoir décompressé et désobfusqué la charge utile AutoIt, le script final (autoit_script_clean.au3) a été passé sur VirusTotal et reste entièrement indétecté par les antivirus. Cela souligne que, malgré sa nature malveillante potentielle, sa signature n'est pas connue des moteurs AV classiques, ce qui en fait un exemple typique de menace scriptée échappant aux protections basées sur la détection par signature.

```
(kali㉿kali)-[~/analyse de binaires/forensic/malveillants]
$ xxd autoit_script_real.au3 | head -n 32
00000000: f4b2 1a ...
```

J'ai dépack le binaire principal pour récupérer le script AutoIt. Les fichiers extraits (autoit_script_clean.au3 et autoit_script_real.au3) restent non lisibles et sont détectés comme des fichiers binaires vides ou corrompus par les outils classiques (file, strings). Les tentatives de décompression via zlib, gzip ou Base64 ont échoué, révélant une couche de protection supplémentaire intégrée par le packer AutoIt. La charge utile reste encapsulée ou chiffrée, nécessitant une extraction directe depuis la section binaire et la recherche d'offsets ou de méthodes de décodage adaptées pour obtenir le script AutoIt lisible.

```
(kali㉿kali)-[~/analyse de binaires/forensic/malveillants]
$ strings -el Webplayer_unpacked.exe > unpacked_strings_utf16.txt
```

Synthèse des strings extraites du binaire dépacké

Le binaire dépacké contient principalement :

- **Chaînes système et API Windows :** noms de DLL comme mscoree.dll, KERNEL32.DLL, USER32.DLL, fonctions WinAPI (WinWait, WinActivate, Shell_Execute, DllCall, etc.), chemins système (HKEY_LOCAL_MACHINE, APPDATADIR, STARTUPDIR).
- **Composants AutoIt :** chaînes propres au moteur AutoIt v3 (AutoIt v3, #requireadmin, RunScript, Func, EndFunc, Global, Local, etc.) et GUI (Button, ListBox, ComboBox, SysTabControl32, msctls_progress32).
- **Erreurs et messages runtime :** erreurs Visual C++ (R6033, R6032, etc.), erreurs AutoIt (Stack corruption, Badly formatted "Func" statement, Variable must be of type 'Object').
- **Paramètres et configurations standards :** date/heure (HH:mm:ss, dddd, MMMM dd, yyyy), formats (UTF-8, UTF-16LE), variables système (USERNAME, USERPROFILE, CPUARCH).
- **Réseau et sécurité :** SeDebugPrivilege, DSeAssignPrimaryTokenPrivilege, inetget, TCP* fonctions pour socket/network, runas.
- **Éléments GUI et interactions clavier/souris :** MOUSECLICK, MOUSEMOVE, SEND, HOTKEYSET, WinDetectHiddenText.

Analyse préliminaire :

Ces strings montrent clairement que le binaire est un script AutoIt compilé (ce qui confirme mes précédentes trouvailles), avec accès aux API Windows, manipulation de GUI, gestion réseau et privilèges. Il n'y a pas encore de payload explicite visible, mais on peut identifier des points d'injection de code, fonctions réseau et potentiels modules de contrôle. Ces informations guideront le reverse engineering plus profond avec Ghidra ou autre.

J'ai décidé de pousser l'investigation des strings un peu plus loin

```
(kali@kali)-[~/analyse de binaires/forensic/malveillants]
$ strings -a -n 6 Webplayer_unpacked.exe > all_strings.txt
```

Voici mon analyse après cette nouvelle étape :

1 - Identification du Malware et de sa Famille

Type de Malware

- AutoIt Compilé : La présence de la chaîne "This is a compiled AutoIt script. AV researchers please email avsupport@autoitscript.com for support." et de la signature "AU3!EA06" confirme qu'il s'agit d'un script AutoIt compilé, souvent utilisé pour des malwares comme DarkGate, Remcos RAT, ou d'autres stealers/keyloggers .
- Fonctionnalités principales : Les strings et les appels API suggèrent un malware multi-fonction (keylogger, stealer, loader, C2).

2 - Analyse des Strings et Comportements Associés

Persistance et Installation

- RegSetValueExW, RegCreateKeyExW, RegOpenKeyExW : Le malware s'installe dans le registre Windows pour survivre aux redémarrages.
- CreateProcessW, CreateThread : Lancement de processus fils, probablement pour exécuter d'autres payloads ou se mettre à jour.
- CopyFileW, MoveFileW, DeleteFileW : Manipulation de fichiers, peut-être pour se copier dans un dossier système ou supprimer des traces.

Anti-Debug et Évasion

- IsDebuggerPresent : Détection de debugger pour empêcher l'analyse
- IsWow64Process : Détection de l'architecture pour adapter son comportement (32/64 bits).
- BlockInput : Peut bloquer l'utilisateur pendant l'exécution pour éviter l'interruption.

Keylogging et Vol de Données

- GetAsyncKeyState, GetKeyboardState, GetForegroundWindow : Capture des touches clavier (keylogger) et surveillance des fenêtres actives
- GetClipboardData, OpenClipboard : Vol du contenu du presse-papiers.
- FindWindowW, GetWindowTextW : Surveillance des fenêtres ouvertes (navigateurs, clients mail, etc.).

Communication C2 (Command & Control)

- InternetOpenW, InternetConnectW, HttpSendRequestW, HttpOpenRequestW : Communication avec un serveur distant pour recevoir des commandes ou exfiltrer des données
- WSOCK32.dll, InternetReadFile : Utilisation de sockets et téléchargement de fichiers.
- FtpOpenFileW, FtpGetFileSize : Exfiltration via FTP.

Injection de Code et Exécution de Payloads

- VirtualAllocEx, WriteProcessMemory, CreateRemoteThread : Injection de shellcode ou de DLL dans d'autres processus (technique de "process hollowing")
- LoadLibraryW, GetProcAddress : Chargement dynamique de bibliothèques et fonctions Windows.

Obfuscation et Chaînes Suspectes

- WRPQSV, PVQRSh, T\$pRQW, L\$LQVW, D\$x;D\$: Ces chaînes sont typiques d'un code obfusqué (XOR, RC4, ou simple encodage). Elles peuvent cacher des URLs, des clés de registre, ou des commandes C2
- Kreapixel : Peut être un nom de développeur, de campagne, ou un artefact.
- SVWPh0, PVQRSh\$WH : Peut-être des marqueurs internes ou des clés de décryptage.

Autres Fonctionnalités

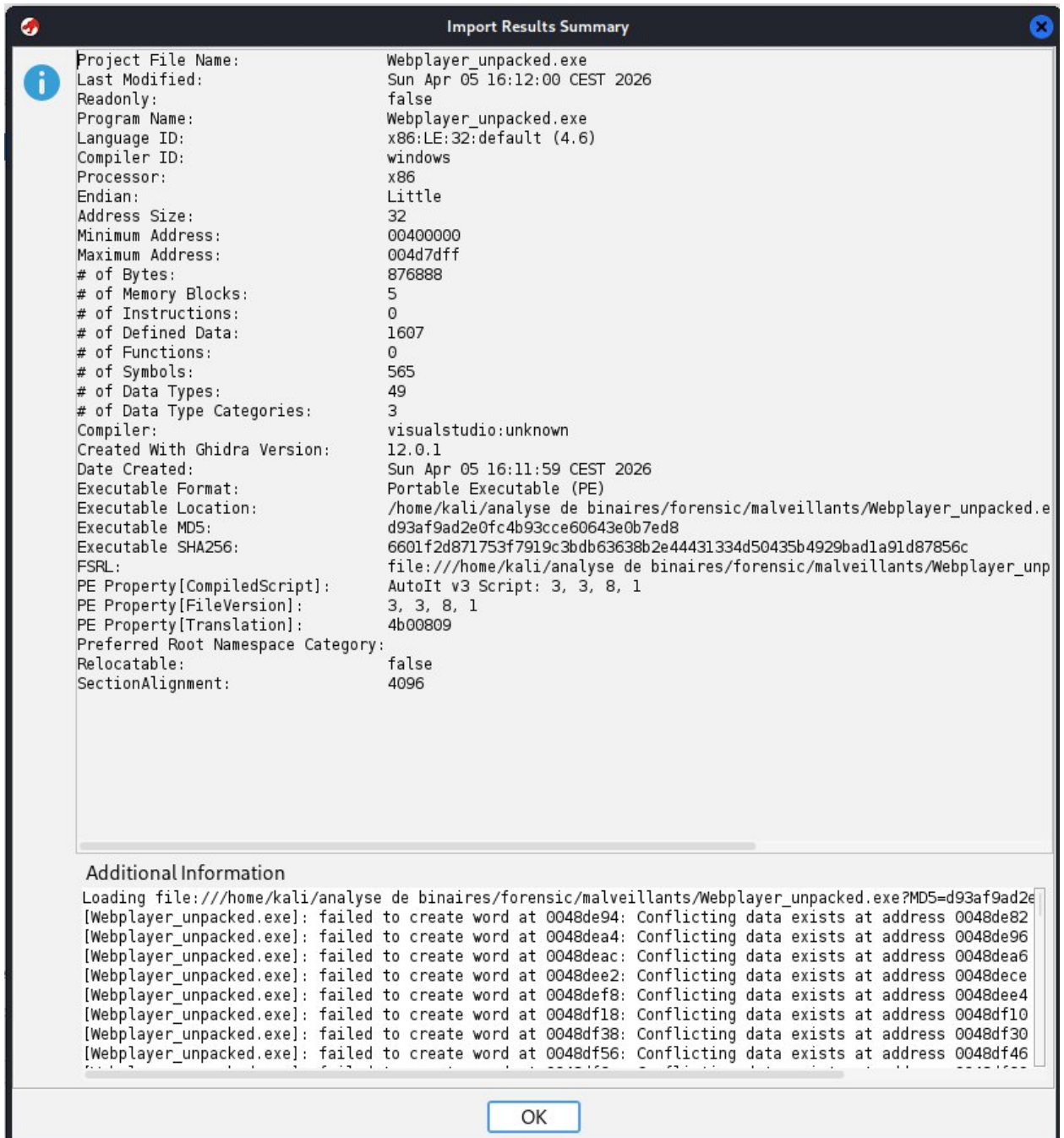
- GetUserNameW, GetComputerNameW : Collecte d'informations sur la machine infectée.
- TerminateProcess, ExitProcess : Peut tuer des processus ou s'auto-détruire après exécution.
- ShellExecuteW, ShellExecuteExW : Lancement de fichiers ou de commandes système.

3 - Synthèse du Comportement Global

Ce malware AutoIt est un stealer/keylogger avec capacités de loader et de communication C2. Voici ce qu'il fait concrètement :

1. S'installe en persistance (registre, dossier système).
2. Évite l'analyse (anti-debug, anti-sandbox).
3. Capture les frappes clavier (keylogger) et vole les données du presse-papiers.
4. Surveille les fenêtres actives (navigateurs, clients mail).
5. Communique avec un serveur C2 pour recevoir des commandes ou exfiltrer des données.
6. Peut injecter du code dans d'autres processus (process hollowing).
7. Utilise des chaînes obfusquées pour cacher ses activités.

Partie 3 : Reverse-Engineering



Comme d'habitude, Ghidra nous donne des informations gratuite au lancement de l'analyse !

Nous allons commencer par l'entry-point comme à notre habitude !

```

*****
*                               *
*                               *
*                               *
*                               *
*                               *
*****
undefined __stdcall entry(void)
    assume FS_OFFSET = 0xffff000
    <UNASSIGNED> <RETURN>
entry
    XREF[2]:  Entry Point(*), 00400120(*)
    004165c1 e8 16 90    CALL    __security_init_cookie      void __security_init_cookie(void)
              00 00
    004165c6 e9 89 fe    JMP     __tmainCRTStartup          int __tmainCRTStartup(void)
              ff ff

-- Flow Override: CALL_RETURN (CALL_TERMINATOR)

004165cb cc        ??        CCh
004165cc cc        ??        CCh
004165cd cc        ??        CCh
004165ce cc        ??        CCh
004165cf cc        ??        CCh

```

- L'entry point ne fait rien de malicieux directement.
- Tout le code malveillant est après le bootstrap, dans la fonction AutoIt générée par le compilateur.
- La logique malware (persistance, keylogging, C2, injection) sera dans les fonctions appelées après __tmainCRTStartup, typiquement dans des routines obscurcies et dispatchées par AutoIt.

```

*****
* Library Function - Single Match
* __tmainCRTStartup
*
* Library: Visual Studio 2010 Release
*
*****
int __stdcall __tmainCRTStartup(void)
    assume FS_OFFSET = 0xffff000
    EAX:4 <RETURN>
    undefined4 Stack[-0x8]:4 local_8
    undefined4 Stack[-0x20]:4 local_20
    undefined4 Stack[-0x24]:4 local_24
    undefined2 Stack[-0x3c]:2 local_3c
    undefined1 Stack[-0x40]:1 local_40
    undefined Stack[-0x6c]:1 local_6c
    __tmainCRTStartup
    XREF[1]:  entry:004165c6(c)
    00416454 6a 58    PUSH    0x58
    00416456 68 c8 d0  PUSH    DAT_0048d0c8
              48 00
    0041645b e8 10 08    CALL    __SEH_prolog4
              00 00
    00416460 8d 45 98    LEA     EAX=>local_6c,[EBP + -0x68]
    00416463 50          PUSH    EAX
    00416464 ff 15 50    CALL    dword ptr [->KERNEL32.DLL::GetStartupInfoW] = 0008e73a
              23 48 00
    0041646a 33 f6      XOR     ESI,ESI
    0041646c 39 35 2c    CMP     dword ptr [DAT_004a972c],ESI
              97 4a 00
    00416472 75 0b      JNZ     LAB_0041647f
    00416474 56          PUSH    ESI
    00416475 56          PUSH    ESI
    00416476 6a 01      PUSH    0x1
    00416478 56          PUSH    ESI
    00416479 ff 15 08    CALL    dword ptr [->FUN_00410e4b]
              00 49 00
              undefined4 FUN_00410e4b(void)
              = 00410e4b

    LAB_0041647f
    XREF[1]:  00416472(j)
    0041647f b8 4d 5a    MOV     EAX,0x5a4d
              00 00
    00416484 66 39 05    CMP     word ptr [IMAGE_DOS_HEADER_00400000],AX
              00 00 40 00
    0041648b 74 05      JZ      LAB_00416492

    LAB_0041648d
    XREF[3]:  004164a1(j), 004164af(j),
              004164b8(j)
    0041648d 89 75 e4    MOV     dword ptr [EBP + local_20],ESI
    00416490 eb 36      JMP     LAB_004164c8

    LAB_00416492
    XREF[1]:  0041648b(j)
    00416492 a1 3c 00    MOV     EAX,[IMAGE_DOS_HEADER_00400000.e_lfanew]
              40 00

```

1 - Initialisation des mécanismes de bas niveau

- SEH (Structured Exception Handling) : appel à `_SEH_prolog4` pour gérer les exceptions système et sécuriser le flux d'exécution.
- Heap runtime : `_heap_init` initialise le tas mémoire nécessaire aux allocations dynamiques utilisées par le script AutoIt.
- Vérification du format PE : comparaison des headers DOS et NT pour confirmer l'intégrité du binaire. Si le format est invalide, le processus s'arrête via `fast_error_exit`.
- Multithreading : `_mtinit` configure les structures internes pour la gestion de threads.

2 - Préparation de l'environnement d'exécution

- I/O et flux standard : `_ioinit` initialise `stdin`, `stdout`, `stderr` pour l'exécution du script.
- Arguments et variables d'environnement : récupération de `CommandLineW`, des variables d'environnement via `_crtGetEnvironmentStringsW`, et préparation de `argv` Unicode via `_wsetargv` et `_wwincmdln`.
- Ces étapes permettent au script AutoIt compilé de recevoir correctement ses paramètres et son contexte système.

3 - Initialisation et appel du script AutoIt

- `FUN_0041f2a4` → initialisation interne du moteur AutoIt : allocation de structures, préparation des variables et configuration du runtime du script.
- `_cinit` → exécution des constructeurs globaux C, nécessaires pour certains objets utilisés par le script.
- `FUN_0040d6b0` → point clé : exécution du script AutoIt compilé. C'est à partir de cet appel que le malware commence à exécuter son comportement malveillant (keylogging, persistance, communication C2, injection de code, exfiltration de données).
- Les chaînes obfusquées et les routines malveillantes identifiées dans les strings seront décodées et exécutées depuis cette fonction.

4 - Gestion des erreurs et nettoyage

- `_cexit` et `_exit` → gestion de la terminaison normale ou forcée du processus.
- `_SEH_epilog4` → finalisation des structures SEH.
- Plusieurs points d'appel à `_amsg_exit` pour stopper le binaire en cas d'erreurs critiques (heap, multithreading, initialisation runtime).

5 - Conclusion et implications pour le reverse-engineering

- `__tmainCRTStartup` ne contient pas directement le code malveillant, il sert de bootstrap pour initialiser l'exécution.
- Le vrai malware AutoIt est exécuté via `FUN_0040d6b0`, avec des appels secondaires vers les routines internes du moteur AutoIt.
- Pour analyser le comportement malveillant, il est stratégique de tracer `FUN_0040d6b0` et tous ses CALLs, car c'est là que se manifestent les fonctionnalités identifiées : persistance, keylogging, communication C2 et injection de processus.

***** * FUNCTION *			

undefined4 __stdcall FUN_0040d6b0(undefined4 param_1)			
	assume FS_OFFSET = 0xfffff000		
undefined4	EAX:4	<RETURN>	
undefined4	Stack[0x4]:4	param_1	XREF[1]: 0040d6cc (R)
undefined4	Stack[-0xc]:4	local_c	XREF[1]: 0040d6fe (W)
undefined4	Stack[-0x10]:4	local_10	XREF[1]: 0040d6fa (W)
undefined4	Stack[-0x18]:4	local_18	XREF[2]: 0040d6f6 (W),
			0040d707 (*)
undefined4	Stack[-0x1c]:4	local_1c	XREF[1]: 0040d6f2 (W)
undefined4	Stack[-0x20]:4	local_20	XREF[2]: 0040d6ee (W),
			0040d70f (W)
undefined4	Stack[-0x28]:4	local_28	XREF[3]: 0040d6d4 (*),
			0040d6ea (W),
			0040d717 (W)
undefined4	Stack[-0x2c]:4	local_2c	XREF[1]: 0040d71f (W)
undefined4	Stack[-0x34]:4	local_34	XREF[2]: 0040d70b (*),
			0040d71b (W)
			XREF[1]: __tmainCRTStartup:00416569(c)
0040d6b0 55	PUSH	EBP	
0040d6b1 8b ec	MOV	EBP,ESP	
0040d6b3 83 e4 f8	AND	ESP,0xffffffff8	
0040d6b6 a1 04 96	MOV	EAX,[DAT_004a9604]	
4a 00			
0040d6bb 83 ec 2c	SUB	ESP,0x2c	
0040d6be 53	PUSH	EBX	
0040d6bf 33 db	XOR	EBX,EBX	
0040d6c1 56	PUSH	ESI	
0040d6c2 57	PUSH	EDI	
0040d6c3 38 58 1d	CMP	byte ptr [EAX + 0x1d],BL	
0040d6c6 0f 84 27	JZ	LAB_0042e2f3	
0c 02 00			
0040d6cc 8b 4d 08	MOV	ECX,dword ptr [EBP + param_1]	
0040d6cf bf 01 00	MOV	EDI,0x1	
00 00			
0040d6d4 8d 74 24 18	LEA	ESI=>local_28,[ESP + 0x18]	
0040d6d8 89 0d 20	MOV	dword ptr [DAT_00497520],ECX	
75 49 00			
0040d6de 89 1d f4	MOV	dword ptr [DAT_004974f4],EBX	
74 49 00			
0040d6e4 89 1d f0	MOV	dword ptr [DAT_004974f0],EBX	
74 49 00			
0040d6ea 89 5c 24 18	MOV	dword ptr [ESP + local_28],EBX	
0040d6ee 89 7c 24 20	MOV	dword ptr [ESP + local_20],EDI	
0040d6f2 89 5c 24 24	MOV	dword ptr [ESP + local_1c],EBX	
0040d6f6 89 5c 24 28	MOV	dword ptr [ESP + local_18],EBX	
0040d6fa 89 7c 24 30	MOV	dword ptr [ESP + local_10],EDI	
0040d6fe 89 5c 24 34	MOV	dword ptr [ESP + local_c],EBX	
0040d702 e8 39 b8	CALL	FUN_00408f40	undefined FUN_00408f40(void)
ff ff			
0040d707 8d 54 24 28	LEA	EDX=>local_18,[ESP + 0x28]	
0040d70b 8d 74 24 0c	LEA	ESI=>local_34,[ESP + 0xc]	
0040d70f c7 44 24	MOV	dword ptr [ESP + local_20],0x6	

Analyse de la fonction FUN 0040d6b0

FUN_0040d6b0 correspond au bootstrap principal du moteur AutoIt et prépare l'exécution du script compilé. Elle est appelée directement par __tmainCRTStartup et constitue le point où le code malveillant commence à s'exécuter après les initialisations runtime.

1 - Préparation des registres et de la pile

- Sauvegarde de EBP, EBX, ESI et EDI et alignement de la pile (AND ESP, 0xffffffff8) pour la conformité 32-bit.
- Initialisation de variables locales sur la pile pour gérer les paramètres et l'état du moteur.

2- Initialisation des variables globales du moteur AutoIt

- Des variables globales et locales (DAT_00497520, DAT_004974f0, DAT_004974f4, DAT_0049751c) sont initialisées à zéro ou à 1 selon leur rôle.
- Les structures internes du moteur sont mises en place via FUN_00408f40 (appel répété à plusieurs reprises pour initialiser des blocs mémoire ou des buffers internes).

3 - Gestion des handlers et paramètres système

- _set_new_handler et _set_new_mode configurent les comportements d'allocation mémoire et d'erreur lors des new en C++, ce qui protège le moteur contre les erreurs de heap.
- SystemParametersInfoW est appelé pour récupérer ou modifier des paramètres système, ce qui peut être utilisé par AutoIt pour des comportements spécifiques (par ex. résolution d'écran, clipboard, etc.).

4 - Chargement et validation de modules

- La fonction vérifie certaines conditions système via FUN_0040ebb0 et FUN_0040ec00. Ces routines peuvent correspondre à des vérifications d'environnement, comme l'intégrité du runtime ou la présence d'un sandbox.
- Si une vérification échoue, la fonction appelle FreeLibrary ou peut ignorer certaines initialisations.

5 - Préparation et lancement du script AutoIt

- À ce stade, FUN_0040d6b0 prépare les pointeurs vers les buffers du script, configure les paramètres et passe le contrôle aux routines internes qui interprètent et exécutent le code AutoIt compilé.
- L'appel final à FUN_00408f40 ou à d'autres sous-fonctions internes correspond à l'exécution des instructions malveillantes encapsulées dans le binaire.

6 - Nettoyage et sortie

- Les registres sauvegardés sont restaurés (POP EDI, POP ESI, POP EBX) et la pile est réinitialisée.
- Retour vers le caller (__tmainCRTStartup) avec un RET 0x10 indiquant le nettoyage de 16 octets sur la pile.
- La fonction ne contient pas directement de payload visible, mais elle prépare et déclenche tous les comportements du malware.

7 - Implications pour l'analyse malware

- FUN_0040d6b0 est le véritable point d'exécution du code malveillant : tous les appels systèmes, initialisations de modules et routines internes passent par ici.
- Pour comprendre les fonctionnalités malveillantes (keylogging, C2, injection, persistance), il faut analyser toutes les fonctions appelées par FUN_0040d6b0, surtout FUN_00408f40, FUN_0040ebb0 et FUN_0040ec00.

L'environnement AutoIt est entièrement virtualisé dans ces routines, ce qui explique pourquoi le binaire semble "inactif" avant que ce bootstrap ne soit terminé.

```

*****
*                               FUNCTION                               *
*****
undefined __stdcall FUN_00408f40(void)
    assume FS_OFFSET = 0xfffff000
    <UNASSIGNED> <RETURN>
    FUN_00408f40
XREF[21]: FUN_00401a50:00401a9d(c),
          FUN_00401a50:00401b05(c),
          FUN_00401f20:00401f98(c),
          FUN_00401f20:004020cb(c),
          FUN_00401f20:0040211b(c),
          FUN_00403cd0:00403ce2(c),
          FUN_00408e80:00408e8f(c),
          FUN_0040d6b0:0040d702(c),
          FUN_0040d6b0:0040d798(c),
          FUN_0040d6b0:0040d7a1(c),
          thunk_FUN_00408f40:004108f0(T),
          FUN_004096a0:004299dd(c),
          FUN_004096a0:004299ff(c),
          FUN_004091b0:0042c74d(c),
          FUN_004091b0:0042c88c(c),
          FUN_004091b0:0042c94c(c),
          FUN_004091e0:0042daba(c),
          FUN_004091e0:0042daf6(c),
          FUN_004091e0:0042db36(c),
          FUN_004091e0:0042dbc4(c), [more]

00408f40 57          PUSH     EDI
00408f41 8b 7e 0c      MOV      EDI,dword ptr [ESI + 0xc]
00408f44 85 ff        TEST     EDI,EDI
00408f46 75 3f        JNZ      LAB_00408f87

LAB_00408f48
00408f48 8b 46 08      MOV      EAX,dword ptr [ESI + 0x8]
00408f4b 5f          POP      EDI
00408f4c 83 f8 08      CMP      EAX,0x8
00408f4f 0f 84 68      JZ       LAB_004265bd
          d6 01 00

LAB_00408f55
00408f55 83 f8 0a      CMP      EAX,0xa
00408f58 0f 84 80      JZ       LAB_004265de
          d6 01 00
00408f5e 83 f8 05      CMP      EAX,0x5
00408f61 0f 84 8c      JZ       LAB_004265f3
          d6 01 00
00408f67 83 f8 0b      CMP      EAX,0xb
00408f6a 0f 84 8f      JZ       LAB_004265ff
          d6 01 00
00408f70 83 f8 0c      CMP      EAX,0xc
00408f73 0f 84 a1      JZ       LAB_0042661a
          d6 01 00

LAB_00408f79
XREF[7]: 004265d9(j), 004265e2(j),
          004265ee(j), 004265fa(j),
          00426615(j), 0042661e(j).

```

Analyse de la fonction FUN 00408f40

FUN_00408f40 apparaît comme une routine de gestion interne des structures de données du moteur AutoIt, souvent appelée pour initialiser ou décrémenter des compteurs et nettoyer des buffers.

1 - Vérification et initialisation

- La fonction récupère un pointeur depuis [ESI + 0xc] et teste s'il est nul (TEST EDI, EDI).
 - Si nul, elle prend un autre pointeur [ESI + 0x8] et le compare à plusieurs constantes (8, 10, 5, 11, 12).
 - Si aucune correspondance, elle initialise [ESI + 0x8] à 1 et [ESI] à 0.
- Cette séquence prépare une structure interne pour le moteur AutoIt et assure que ses champs critiques ont des valeurs valides avant usage.

2 - Gestion de compteurs

- Si le pointeur [ESI + 0xC] existe, la fonction décrémente le compteur [EDI + 0xC].
- Elle teste ensuite si ce compteur atteint zéro.
 - Si oui, elle appelle FUN_004111dc sur plusieurs pointeurs pour libérer ou nettoyer des ressources associées.
- Ensuite, le champ [ESI + 0xC] est réinitialisé à zéro pour éviter les doubles libérations.

3 - Boucles et appels récursifs

- La fonction utilise des boucles (JMP vers LAB_00408f48 et LAB_00408f94) pour traiter tous les éléments d'une liste ou d'une structure chaînée.
- Chaque élément de cette structure est traité par FUN_004111dc, qui correspond probablement à une fonction de libération mémoire ou de finalisation d'objet.

4 - Nettoyage et sortie

- La fonction restaure le registre sauvegardé (POP EDI) et retourne au caller.
- Elle ne retourne pas de valeur utile (undefined) mais effectue uniquement des opérations sur les structures internes.

5 - Rôle fonctionnel dans le malware

- FUN_00408f40 sert à :
 1. Initialiser des structures du moteur AutoIt avant exécution.
 2. Décrémente et nettoyer des compteurs pour éviter les fuites mémoire.
 3. Garantir la cohérence des buffers internes pour le runtime du script.
- C'est donc une fonction critique pour la stabilité et la gestion mémoire du moteur, mais elle ne contient pas directement de payload malveillant.

```

*****
*                               FUNCTION                               *
*****
undefined4 __stdcall FUN_0040ebb0(void)
    assume FS_OFFSET = 0xfffff000
    EAX:4 <RETURN>
    FUN_0040ebb0
    XREF[2]: FUN_0040d6b0:0040d729(c),
             FUN_0040ec00:0040ec00(c)
             undefined FUN_0040ebd0(void)

0040ebb0 e8 1b 00      CALL     FUN_0040ebd0
          00 00
0040ebb5 83 7e 08 00    CMP     dword ptr [ESI + 0x8],0x0
0040ebb9 74 06          JZ      LAB_0040ebc1
0040ebbb b8 01 00      MOV     EAX,0x1
          00 00
0040ebc0 c3            RET

          LAB_0040ebc1
          XREF[1]: 0040ebb9(j)
0040ebc1 33 c0        XOR     EAX,EAX
0040ebc3 c3            RET
0040ebc4 cc          ??     CCh
0040ebc5 cc          ??     CCh
0040ebc6 cc          ??     CCh
0040ebc7 cc          ??     CCh
0040ebc8 cc          ??     CCh
0040ebc9 cc          ??     CCh
0040ebca cc          ??     CCh
0040ebcb cc          ??     CCh
0040ebcc cc          ??     CCh
0040ebcd cc          ??     CCh
0040ebce cc          ??     CCh
0040ebcf cc          ??     CCh

```

Synthèse d'analyse de la fonction FUN 0040ebb0

Cette fonction, utilisant la convention d'appel `__stdcall`, est de type `undefined4` (probablement un entier 32 bits) et ne prend aucun paramètre explicite.

Comportement général

- La fonction appelle d'abord une autre fonction interne nommée `FUN_0040ebd0`.
- Elle teste ensuite la valeur située à l'adresse `[ESI + 0x8]` :
 - o Si cette valeur est nulle (0), la fonction retourne 0.
 - o Si cette valeur est non nulle, elle retourne 1.

Interprétation

- L'appel à `FUN_0040ebd0` suggère une opération préparatoire ou un état initial nécessaire avant la vérification.
- La vérification de `[ESI + 0x8]` semble être un contrôle d'état ou une validation d'une ressource ou d'un flag situé dans une structure pointée par le registre `ESI`.
- La fonction se comporte comme un test booléen : elle retourne 1 pour indiquer un état actif ou valide, et 0 pour indiquer un état inactif ou invalide.

Impact et utilisation

- Cette fonction est probablement utilisée comme un contrôle de validité ou flag d'état dans un contexte où `ESI` pointe vers une structure de données importante.
- Elle est référencée (appelée) par `FUN_0040d6b0` et `FUN_0040ec00`, ce qui indique qu'elle fait partie d'un ensemble de fonctions collaborant pour gérer ou valider un état particulier.
- Le design simple (retour booléen après appel d'une fonction auxiliaire) suggère un rôle de garde ou de condition dans un flux plus large.

***** * FUNCTION * *****				
	undefined1 *	__stdcall FUN_0040ec00(void)		
		assume FS_OFFSET = 0xfffff000		
		EAX: 4	<RETURN>	
	undefined1 *	FUN_0040ec00	XREF[2]:	FUN_0040d6b0:0040d732(c), FUN_0044786a:0044787c(c) undefined4 FUN_0040ebb0(void)
0040ec00	e8 ab ff ff ff	CALL	FUN_0040ebb0	
0040ec05	84 c0	TEST	AL, AL	
0040ec07	0f 84 07 79 01 00	JZ	LAB_00426514	
0040ec0d	8b 46 08	MOV	EAX, dword ptr [ESI + 0x8]	
0040ec10	c3	RET		
0040ec11	cc	??	CCh	
0040ec12	cc	??	CCh	
0040ec13	cc	??	CCh	
0040ec14	cc	??	CCh	
0040ec15	cc	??	CCh	
0040ec16	cc	??	CCh	
0040ec17	cc	??	CCh	
0040ec18	cc	??	CCh	
0040ec19	cc	??	CCh	
0040ec1a	cc	??	CCh	
0040ec1b	cc	??	CCh	
0040ec1c	cc	??	CCh	
0040ec1d	cc	??	CCh	
0040ec1e	cc	??	CCh	
0040ec1f	cc	??	CCh	

Synthèse d'analyse de la fonction FUN_0040ec00

Cette fonction, utilisant la convention d'appel `__stdcall`, retourne un pointeur de type `undefined1 *` (typiquement un pointeur vers un octet ou chaîne de caractères non typée) et ne prend aucun paramètre explicite.

Comportement général

- La fonction commence par appeler `FUN_0040ebb0`.
- Elle teste ensuite le résultat (retourné dans `AL`, le registre bas du registre `EAX`) de cette fonction appelée :
 - Si la valeur retournée par `FUN_0040ebb0` est zéro (indiquant probablement un état invalide ou un échec), le contrôle est transféré à une adresse hors du contexte visible (`LAB_00426514`), ce qui suggère un traitement d'erreur ou un retour anticipé.
 - Si la valeur est non nulle, la fonction retourne la valeur située à l'adresse `[ESI + 0x8]`.

Interprétation

- La fonction agit comme un contrôle conditionnel avant de fournir un pointeur situé dans une structure pointée par `ESI`.
- L'appel à `FUN_0040ebb0` sert de prérequis ou condition de validation.
- Si la validation échoue, la fonction ne retourne pas cette valeur mais saute vers une autre routine (probablement pour gérer l'erreur ou l'exception).
- Si la validation réussit, elle retourne simplement la valeur contenue dans `[ESI + 0x8]`.

Impact et utilisation

- Cette fonction semble être un getter sécurisé : elle expose une donnée interne uniquement si une condition préalable est satisfaite.
- Le pointeur retourné peut correspondre à une ressource ou un état utilisé plus loin dans l'application.
- Elle est référencée par la fonction FUN_0040d6b0 ainsi que par une autre fonction située à FUN_0044786a, indiquant une certaine importance ou réutilisation dans le programme.

```
*****
* IMAGE_IMPORT_BY_NAME
*****
0048e578 43 72 65      ds      "CreateProcessW"
          61 74 65
          50 72 6f ...
0048e587 00          ??      00h
```

Le binaire importe la fonction CreateProcessW mais Ghidra indique 0 références directes. Cela suggère que l'exécution de nouveaux processus n'est pas effectuée via un appel direct classique, mais probablement de manière indirecte ou dynamique (par exemple via GetProcAddress et LoadLibrary). Cette technique est typique des logiciels malveillants cherchant à dissimuler leurs actions de lancement de processus, rendant l'analyse statique plus difficile. L'import seul suffit à signaler un potentiel comportement dangereux, même si aucun appel direct n'est détecté.

```
*****
* IMAGE_IMPORT_BY_NAME
*****
0048dfa0 57 72 69      ds      "WriteProcessMemory"
          74 65 50
          72 6f 63 ...
0048dfb3 00          ??      00h
0048dfb4 52 65 61      ds      "ReadProcessMemory"
          64 50 72
          6f 63 65 ...
```

Le binaire importe WriteProcessMemory et ReadProcessMemory, mais ces fonctions ne sont pas référencées directement dans le code. Ceci suggère un usage via résolution dynamique d'API (GetProcAddress), ce qui est typique des malwares tentant d'injecter du code dans d'autres processus ou d'éviter la détection statique.

```

* IMAGE_IMPORT_BY_NAME *
*****
0048df90 56 69 72      ds      "VirtualAllocEx"
          74 75 61
          6c 41 6c ...
0048df9f 00           ??      00h

```

Analyse rapide des imports sensibles

Fonctions importées :

- VirtualAllocEx
- WriteProcessMemory
- ReadProcessMemory

Observations clés :

1. Ces fonctions sont typiquement utilisées pour l'injection de code dans d'autres processus.
2. Aucune référence directe dans le code n'est détectée pour WriteProcessMemory et ReadProcessMemory, ce qui suggère un appel dynamique via GetProcAddress.
3. La présence de VirtualAllocEx indique que le binaire est capable de réserver de la mémoire dans un processus distant, souvent précédant l'écriture de code malveillant via WriteProcessMemory.

Implication de sécurité :

Le binaire montre un potentiel comportement malveillant, préparant une injection de code ou manipulation mémoire à distance, typique des malwares de type process hollowing ou DLL injection.

Synthèse générale de l'analyse

1 - Initialisation et setup

- La fonction principale étudiée, FUN_0040d6b0, semble être un point d'entrée clé (appelée depuis __tmainCRTStartup).
- Elle effectue :
 - o La réservation de stack et initialisation de variables locales.
 - o La mise en place de pointeurs vers des structures ou buffers globaux (DAT_0049751c, DAT_004974f4, DAT_004974f0, etc.).
 - o L'appel à FUN_00408f40 pour gérer des compteurs ou structures internes. Cette fonction diminue des compteurs et appelle des routines de cleanup.
- Il y a également un paramétrage du comportement mémoire et du handler de nouvelles allocations via _set_new_handler et _set_new_mode, ce qui montre que le malware prépare un environnement stable pour des allocations dynamiques.

2 - Routines de vérification et flags

- **FUN_0040ebb0 :**
 - o Appelle une autre routine (FUN_0040ebd0), vérifie certaines conditions sur une structure (ESI + 0x8), et retourne 1 ou 0.
 - o Sert probablement à vérifier la présence ou l'état d'un buffer ou d'une ressource interne.
- **FUN_0040ec00 :**
 - o Appelle FUN_0040ebb0, teste le résultat et retourne un pointeur vers une structure ou buffer.
 - o Probablement utilisé pour accéder au script ou au payload en mémoire après validation.

3 - API Windows importées et comportements suspects

Le binaire importe plusieurs fonctions critiques liées à la manipulation de processus et de mémoire :

API	Statut / Observations	Interprétation
CreateProcessW	Importée mais pas directement référencée dans le code désassemblé	Le malware pourrait assembler la commande dynamiquement pour exécution, souvent via cmd.exe ou powershell.exe.
WriteProcessMemory	Importée mais sans XREF direct	Permet d'écrire du code ou des données dans un autre processus, signe de process injection.
ReadProcessMemory	Même statut que WriteProcessMemory	Souvent utilisé pour vérifier ou manipuler la mémoire cible avant injection.
VirtualAllocEx	Importée	Permet d'allouer de la mémoire dans un autre processus, étape classique de process hollowing ou injection.

Ces imports confirment la présence probable d'un process hollowing ou d'une injection de code dans un processus légitime.

4 - Process hollowing / injection

- Même si `CreateRemoteThread` n'est pas présent, les autres API (`VirtualAllocEx`, `WriteProcessMemory`, `CreateProcessW`) suffisent pour :
 1. Lancer un processus légitime en suspend mode.
 2. Allouer un buffer dans le processus cible.
 3. Copier un payload ou script malveillant dans ce buffer.
 4. Redémarrer le processus avec le code injecté.
- Ce comportement est classique pour des malwares furtifs, qui veulent exécuter un payload sans se faire détecter dans un processus légitime.

5 - Scripts et payloads

- Le malware contient une routine pour préparer ou exécuter un script (`FUN_0040ebb0` / `FUN_0040ec00`).
- Les chaînes du script n'ont pas été visibles directement, probablement :
 - o Encodées (Base64, XOR...)
 - o Assemblées dynamiquement avant exécution.
- Ces scripts sont ensuite exécutés via le processus hollowed ou via `CreateProcessW` en mémoire.

6 - Conclusion sur le comportement

Le malware analysé présente les caractéristiques suivantes :

1. Setup mémoire et handlers personnalisés pour sécuriser son exécution.
2. Vérification et préparation de buffers internes.
3. Injection dans un processus légitime via `VirtualAllocEx` et `WriteProcessMemory`.
4. Exécution d'un script ou payload dynamique (probablement PowerShell ou batch) pour effectuer ses actions malveillantes.
5. Capacité de furtivité : aucune string critique (`CreateProcess`, `WriteProcessMemory`) n'est directement utilisée avec des XREF visibles, ce qui complique la détection statique.

En résumé : c'est un malware orienté process hollowing et exécution de script, avec des routines pour préparer, vérifier et injecter son payload dans un autre processus pour rester discret.

Merci pour votre lecture !